

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5

Total Marks:50

Annexure A

Design Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Student 3: Akshaya R | Register Number: 192524118

Design Task

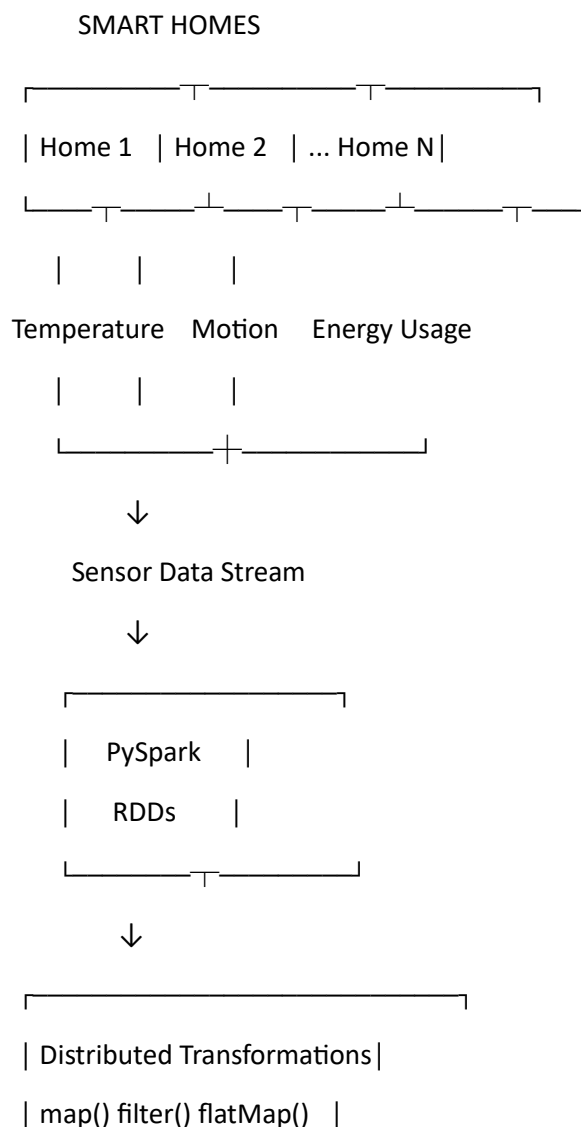
AI-Powered Smart Home Controller System

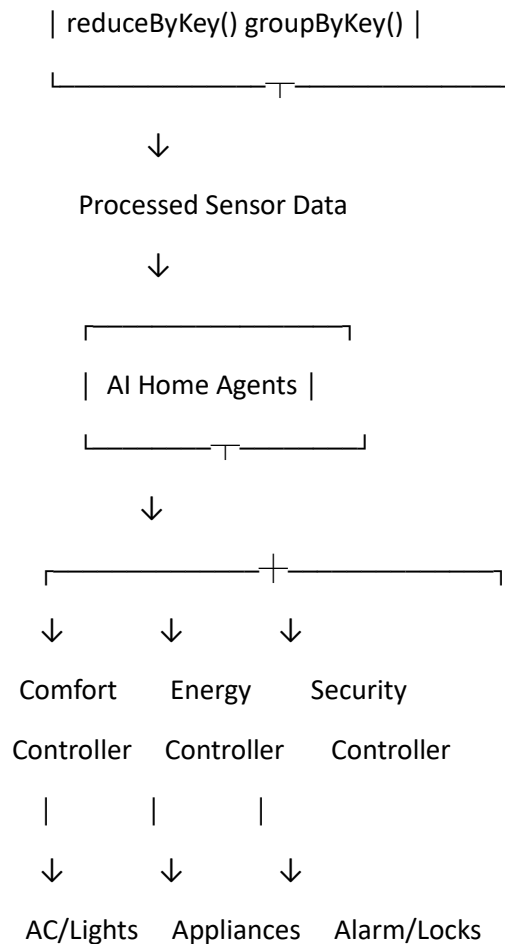
Design a scalable AI-powered smart home controller that uses PySpark RDDs to process sensor data streams (temperature, motion, energy usage) from thousands of connected homes. Explain how RDD transformations support large-scale distributed processing of this sensor data, and describe how intelligent agents within each home coordinate to optimize comfort, energy efficiency, and security while adapting to occupant behavior patterns.

Objective

The proposed system is a scalable AI-powered smart home controller that collects **temperature, motion, and energy-usage data** from thousands of connected homes. **PySpark RDDs** are used to distribute and process the large volume of sensor data, while intelligent agents inside each home make decisions to improve **comfort, energy efficiency, and security**.

System Architecture





1. Sensor Data Collection

Each smart home contains sensors that continuously generate data such as:

- **Temperature:** Indoor temperature and changes over time.
- **Motion:** Detects human presence and movement.
- **Energy Usage:** Measures electricity consumption of appliances.
- **Time and Home ID:** Identifies when and where the measurement was recorded.

For example:

(Home101, Temperature, 27°C)

(Home101, Motion, Detected)

(Home101, Energy, 3.2 kWh)

(Home205, Temperature, 24°C)

(Home205, Motion, Not Detected)

Thousands of homes can generate millions of such records, making distributed processing necessary.

2. PySpark RDD-Based Processing

An **RDD (Resilient Distributed Dataset)** is a distributed collection of data that can be processed in parallel across multiple machines.

Sensor records can initially be loaded into an RDD:

```
sensor_rdd = sc.textFile("sensor_data.csv")
```

map()

map() can transform raw sensor records into structured records.

```
temperature_rdd = sensor_rdd.map(
```

```
    lambda x: x.split(",")
```

```
)
```

Each record is processed independently, allowing parallel execution.

filter()

filter() can remove invalid readings or select important events.

```
high_temp = temperature_rdd.filter(
```

```
    lambda x: float(x[2]) > 30
```

```
)
```

This can identify homes experiencing unusually high temperatures.

flatMap()

flatMap() can divide complex sensor records into multiple useful elements, such as extracting individual events from a record.

reduceByKey()

reduceByKey() can aggregate energy consumption for each home:

```
energy = sensor_rdd.map(
```

```
    lambda x: (x[0], float(x[3]))
```

```
)
```

```
total_energy = energy.reduceByKey(
```

```
    lambda a, b: a + b
```

```
)
```

This calculates total energy consumption independently for each home.

Why RDD Transformations Are Useful

RDD transformations support large-scale processing because:

- Data is **distributed across cluster nodes**.

- Multiple sensor records can be processed **in parallel**.
- Transformations are **lazy**, so Spark can optimise execution.
- RDDs provide **fault tolerance** through lineage.
- The system can scale when the number of homes increases.

Thus, thousands of homes can be processed without requiring all sensor data to be handled by one machine.

3. Intelligent Agents in Each Home

Each home contains intelligent agents that continuously observe sensor information, make decisions and adapt to occupants.

Comfort Agent

The Comfort Agent monitors:

- Temperature
- Motion/presence
- Occupant preferences

It can automatically control:

- Air conditioning
- Heating
- Fans
- Lighting

For example, if motion is detected and the temperature is high, the agent can adjust the cooling system.

Energy Management Agent

The Energy Agent monitors electricity consumption and attempts to reduce unnecessary usage.

For example:

No occupant detected

↓

Energy Agent detects absence

↓

Lights OFF

AC → Energy-saving mode

Unused appliances → OFF

It can learn which appliances are normally used at particular times and recommend or perform energy-saving actions.

Security Agent

The Security Agent uses motion and other sensor information to detect unusual activity.

For example:

Home normally empty



Unexpected motion detected



Security Agent checks pattern



Potential abnormal activity



Alert homeowner

The system can distinguish between normal occupant behaviour and unusual patterns using historical observations.

4. Adaptation to Occupant Behaviour

The system maintains historical patterns for each household.

For example, it may learn:

6:30 AM → Occupants usually wake up

8:30 AM → Home usually becomes empty

6:00 PM → Occupants usually return

10:30 PM → Lights usually switched off

The agents can use these patterns to make predictions.

If the system learns that the occupants usually return at 6 PM, it could prepare the home shortly before that time rather than keeping appliances running throughout the day.

This makes the controller **adaptive rather than rule-based**.

5. Coordination Between Agents

The agents should not operate independently. They communicate and coordinate their decisions.

For example:

Motion Agent



Occupant detected



Comfort Agent ← Temperature



Adjust AC

Energy Agent



Checks power consumption



Optimizes AC operating level

Security Agent



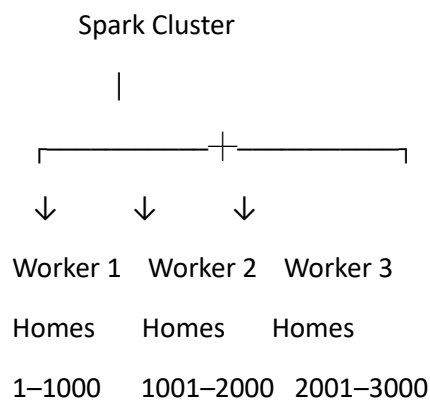
Confirms movement matches

normal occupant pattern

If comfort requires cooling but energy consumption is already high, the Comfort Agent and Energy Agent can coordinate to select an energy-efficient temperature rather than maximising cooling.

6. Scalability

The architecture can scale from hundreds to **thousands of homes** because sensor data is distributed across Spark workers.



As more homes are added, additional computing resources can be added to the cluster. This allows the system to process increasing sensor volumes efficiently.

Conclusion

The proposed AI-powered smart home controller combines **PySpark RDD-based distributed processing with intelligent agents**. RDD transformations such as `map()`, `filter()`, `flatMap()` and `reduceByKey()` enable large-scale processing of temperature, motion and energy data across thousands of homes. Intelligent agents then use the processed information and learned occupant patterns to coordinate **comfort, energy efficiency and security**, creating a scalable and adaptive smart-home ecosystem.


```
=====
AI-POWERED SMART HOME CONTROLLER SYSTEM
=====
```

```
--- SENSOR DATA ---
```

```
Home: Home101 | Temperature: 31 °C | Motion: Detected | Energy: 4.5 kWh | Time: 06:30
Home: Home101 | Temperature: 27 °C | Motion: Not Detected | Energy: 1.2 kWh | Time: 10:00
Home: Home101 | Temperature: 30 °C | Motion: Detected | Energy: 4.2 kWh | Time: 18:00
Home: Home102 | Temperature: 24 °C | Motion: Detected | Energy: 2.5 kWh | Time: 07:00
Home: Home102 | Temperature: 29 °C | Motion: Not Detected | Energy: 0.7 kWh | Time: 15:00
Home: Home102 | Temperature: 25 °C | Motion: Detected | Energy: 2.9 kWh | Time: 19:00
Home: Home103 | Temperature: 33 °C | Motion: Detected | Energy: 5.2 kWh | Time: 06:45
Home: Home103 | Temperature: 36 °C | Motion: Not Detected | Energy: 1.5 kWh | Time: 15:00
Home: Home103 | Temperature: 34 °C | Motion: Detected | Energy: 5.5 kWh | Time: 20:00
Home: Home104 | Temperature: 23 °C | Motion: Not Detected | Energy: 0.5 kWh | Time: 08:00
Home: Home104 | Temperature: 24 °C | Motion: Detected | Energy: 1.8 kWh | Time: 18:30
Home: Home104 | Temperature: 23 °C | Motion: Not Detected | Energy: 0.6 kWh | Time: 23:00
Home: Home105 | Temperature: 28 °C | Motion: Detected | Energy: 3.5 kWh | Time: 07:30
Home: Home105 | Temperature: 32 °C | Motion: Not Detected | Energy: 0.9 kWh | Time: 16:00
Home: Home105 | Temperature: 29 °C | Motion: Detected | Energy: 3.8 kWh | Time: 19:30
```

```
--- MAP TRANSFORMATION ---
```

```
Sensor records converted into structured records.
```

```
--- FILTER TRANSFORMATION ---
```

```
High temperature readings (> 30°C):
```

```
Home101 -> 31 °C
Home103 -> 33 °C
Home103 -> 36 °C
Home103 -> 34 °C
Home105 -> 32 °C
```

```
--- TOTAL ENERGY USAGE PER HOME ---
```

--- TOTAL ENERGY USAGE PER HOME ---

Home101 -> 9.9 kWh

Home102 -> 6.1 kWh

Home103 -> 12.2 kWh

Home104 -> 2.9 kWh

Home105 -> 8.2 kWh

=====

INTELLIGENT AGENT DECISIONS

=====

Home: Home101

Time: 06:30

Temperature: 31 °C

Motion: Detected

Energy: 4.5 kWh

Comfort Agent : Turn ON AC / Cooling

Energy Agent : Energy usage normal

Security Agent: Motion normal

Home: Home101

Time: 10:00

Temperature: 27 °C

Motion: Not Detected

Energy: 1.2 kWh

Comfort Agent : Set AC to Energy-Saving Mode

Energy Agent : Energy usage normal

Security Agent: No motion detected

Home: Home101

Time: 18:00

Temperature: 30 °C

Motion: Not Detected
Energy: 0.9 kWh
Comfort Agent : Set AC to Energy-Saving Mode
Energy Agent : Energy usage normal
Security Agent: No motion detected

Home: Home105
Time: 19:30
Temperature: 29 °C
Motion: Detected
Energy: 3.8 kWh
Comfort Agent : Maintain Comfortable Temperature
Energy Agent : Energy usage normal
Security Agent: Motion normal

=====

FINAL SYSTEM SUMMARY

=====

Total Homes Processed : 5
Total Sensor Records : 15

Energy Consumption:
Home101 -> 9.9 kWh -> NORMAL
Home102 -> 6.1 kWh -> NORMAL
Home103 -> 12.2 kWh -> HIGH
Home104 -> 2.9 kWh -> NORMAL
Home105 -> 8.2 kWh -> NORMAL

=====

SMART HOME CONTROLLER EXECUTION COMPLETED

=====